Phong, Casper, Tino, Emil, Verner

PlantSnap Technical Documentation

Metropolia Ammattikorkeakoulu

Insinööri (AMK)

Tieto- ja viestintätekniikka

Technical Documentation

02.05.2026

Sisällys

1. Introduction **1**
1.1 Core user flows 1
2. Architecture **1**
2.1 Pattern - MVVM 1
2.2 Package structure 2
2.3 State Management 3
2.4 Dependency injection 3
3. Local Database **3**
3.1 Overview 3
3.2 Entities 4
4. Supabase **4**
4.1 Tables 4
4.2 Row Level Security 5
4.3 Authentication 5
4.4 Supabase storage 5
5. External API and SDK integrations **5**
5.1 PlantNet API 5
5.2 Gemini AI API 6
5.3 Wikipedia API 6
5.4 Google Play Services Location 7
5.5 Maps Compose SDK 7
6. Key Components **7**
6.1 PlantService 7
6.2 PlantDetailViewModel 8
6.3 SettingsRepository 8
6.4 PlantOfTheDayHolder 8
6.5 SafetyAdvisor 8
7. Testing **8**
7.1 Test Stack 8
7.2 Unit Test coverage 9
7.3 Running Tests 10
7.4 Static Code Analysis - SonarCloud 11
8. Build and Setup **11**
8.1 Prerequisites 11
8.2 API keys (local.properties) 12
8.3 Supabase setup 12
8.4 Build configuration 12
8.5 Key dependencies 13
9 Continuous integration **13**

1. Introduction

PlantSnap is an Android application that lets users take a photo of any plant and receive an instant species identification, with care guides, habitat information and toxicity warnings. It was developed over six sprints as a mobile development project by a team of Metropolia UAS software engineering students.

This document covers the application architecture, database schema, key technical decisions, build setup, testing strategy and API integration.

1.1 Core user flows

- Take a photo of a plant → organ selection → PlantNet species identification → ranked candidate list
- Tap a candidate → Gemini generated care guide, habitat, toxicity and safety information
- Sign in with Google SSO → scans synced to Supabase cloud
- Save plants to My Garden → view collection → care routine tracking
- Plant of the Day → daily cached plant suggestion with Gemini-generated details
- Configure theme, temperature units

2. Architecture

2.1 Pattern - MVVM

PlantSnap follows the MVVM (Model-View-ViewModel) pattern, with Hilt dependency injection. Compose UI observes StateFlow values exposed by ViewModels. ViewModels delegate work to services and Repositories. Repositories abstract both local Room storage and remote APIs.

Layer	Responsibility	Key classes
View (Compose)	Render UI, collect StateFlow, dispatch user events	HomeScreen, PlantDetailScreen, SettingsScreen, MyGardenScreen
ViewModel	Hold UI state as StateFlow, coordinate business logic	HomeViewModel, PlantDetailViewModel, SettingsViewModel, AuthViewModel
Service	Orchestrate cross-repository operations	PlantService
Repository	Abstract data sources - Room, Supabase, APIs	ScanRepository, GeminiRepository, SettingsRepository, ProfileRepository
Data source	Room database, Supabase client, Retrofit APIs, DataStore	PlantSnapDatabase, SupabaseClient, PlantNetApi, DataStore

2.2 Package structure

```
app/
└── src/main/java/com/plantsnap/
    ├── data/
    │   ├── local/          # Room database — ScanEntity, ScanDao, PlantSnapDatabase
    │   ├── remote/         # Retrofit services — PlantNetService, GeminiService
    │   └── supabase/       # Supabase auth and Postgrest repositories
    ├── domain/
    │   └── models/         # Domain models — ScanResult, PlantMatch
    ├── di/                 # Hilt dependency injection modules
    ├── navigation/         # Compose NavHost and route definitions
    └── ui/
        ├── screens/
        │   ├── home/       # HomeScreen + HomeViewModel
        │   ├── identify/
        │   │   └── camera/ # CameraScreen, CameraViewModel, CaptureButton, ShutterFlash
        │   ├── results/    # ResultsScreen + IdentificationViewModel
        │   ├── history/    # HistoryScreen
        │   ├── profile/    # ProfileScreen
        │   └── auth/       # Login + Register screens
        ├── state/          # UiState sealed class (Idle, Loading, Success, Error)
        └── theme/          # Material 3 Digital Herbarium colour scheme + typography
```

2.3 State Management

All Ui state is held as StateFlow in ViewModels and collected with collectAsStateWithLifecycle() in composables

```
sealed class UiState<out T> {
    object Idle    : UiState<Nothing>()
    object Loading : UiState<Nothing>()
    data class Success<T>(val data: T) : UiState<T>()
    data class Error(val message: String) : UiState<Nothing>()
}
```

2.4 Dependency injection

Hilt is used throughout the application, two hilt modules handle all the bindings.

1. DatabaseModule - provides Room database instance and all DAOs
2. SupabaseModule - provides the SupabaseClient and binds all repository interfaces to their implementations

3. Local Database

3.1 Overview

PlantSnapDatabase is a Room database (SQLite) that persists all scan data locally. Supabase is used for cloud sync and auth.

3.2 Entities

Entity	Table	Purpose
ScanEntity	scans	One row per plant identification
CandidateEntity	candidates	Ranked species candidates from PlantNet
SavedPlantEntity	saved_plants	Plants saved to My Garden
PlantOfTheDayEntity	plant_of_the_day	Single-row daily cache

4. Supabase

Supabase provides authentication, PostgresSQL cloud storage, and file storage for PlantSnap.

4.1 Tables

Table	Purpose	Primary Key
profiles	User profile and onboarding preferences	user_id (FK auth.users.id)
settings	Per-user app settings	user_id (FK auth.users.id)
plant_details	Cached Gemini AI plant info	plant_gbif_id
saved_plants	My Garden collection	id (uuid)
scans	Cloud copy of plant identification history	id (uuid)

4.2 Row Level Security

All five tables have RLS enabled. The policy on every table restricts access to the row owner:

```
-- Applied to profiles, settings, scans, saved_plants
create policy "Users manage own data"
  on public.<table_name>
  for all
  using (auth.uid() = user_id)
  with check (auth.uid() = user_id);

-- plant_details is shared across users — readable by all authenticated users
create policy "Authenticated users read plant details"
  on public.plant_details for select
  using (auth.role() = 'authenticated');
```

4.3 Authentication

Authentication is handled by Supabase Auth. Google Sign in - via Android Credential Manager, idToken passed to Supabase. Session state is observed from supabaseClient.auth.sessionStatos in AuthViewModel.

4.4 Supabase storage

Plant images for My Garden are uploaded to a Supabase Storage bucked (saved_plants). The imageUrl is stored in saved_plants.image_url and displayed via Coil in the garden.

5. External API and SDK integrations

5.1 PlantNet API

PlantNet is used for species identification. The user selects one or more photos and an organ type. The app sends a multipart request to PlantNets /identify endpoint and receives a ranked list of species matches.

Field	Detail
Base URL	https://my-api.plantnet.org/v2/
Endpoint	POST /identify/{project}
Auth	API key via query parameter
Response	List of Results objects with species, score, IUCN category, GBIF ID
Error codes	404 - no match found, 401 - invalid key, 429 - rate limit exceeded

5.2 Gemini AI API

The Gemini API is used in two flows, fetching structured plant detail information, and recommending a daily Plant of the day. All calls go through GeminiRepositoryImpl, which builds personalised prompts from the user's profile.

Flow	Prompt	Output model
getPlantInfo(name)	petType : toxicity detail, experienceLevel: care language, plantInterests: care context, temperature unit: C or F	PlantAiInfo(care, toxicity, safety, habitat, description)
getPlantOfTheDay()	plantInterests: interest hint, experience level: tone, temperature unit: C or F	PlantOfTheDay(scientificName, commonName, care, toxicity, habitat, description, imageUrl)

5.3 Wikipedia API

After fetching Plant of the Day from Gemini, GeminiRepositoryImpl looks up a representative image from the Wikipedia summary APi. This gives high-quality , freely licensed plant images.

5.4 Google Play Services Location

This handles acquiring the device's GPS coordinates. At scan time, PlantService calls FusedLocationProviderClient.getCurrentLocation() with Priority.PRIORITY_HIGH_ACCURACY. If ACCESS_FINE_LOCATION or ACCESS_COARSE_LOCATION is granted, the resulting latitude/longitude pair is stored alongside the scan in Room. If permission is denied, location is stored as null and no map is shown.

5.5 Maps Compose SDK

Maps SDK is used for rendering two map sections: a scan location map showing where the device was when the plant was identified (using GPS coordinates stored in Room), and a natural habitat map with up to two markers at coordinates returned by the Gemini API representing the species' native habitat

6. Key Components

6.1 PlantService

PlantService is the domain-layer controller. It is injected into ViewMOdels and coordinates across repositories. Key methods:

- identifyPlantAndSaveToLocal() - calls PlantNet, maps response to ScanResult and saves to Room
- requestAdditionalInfo() - calls Gemini for plant detail, serialises to JSON, saves to CandidateEntity.aiInfo for caching
- getPlantOfTheDay() - cache-first, checks Room PlantOfTheDayDao for today's date, on miss fetches from Gemini and caches

6.2 PlantDetailViewModel

Loads a scan candidate by scanId + candidateIndex from ScanRepository. Checks the candidates aiInfo field for a cached JSON blob, if present, deserialises and displays without any network calls. If absent or malformed, call PlantService.requestAdditionalInfo().

6.3 SettingsRepository

Implements a cache-first using DataStore as the local cache and Supabase as the remote store. On updateSettings(), DataStore is written first, then Supabase is updated asynchronously. On failure, the DataStore value is retained and will re-sync on next launch, observeSettings() returns Flow<UserSettings> from DataStore.

6.4 PlantOfTheDayHolder

A @Singletion in-memory holder that bridges HomeViewModel and PlantOfTheDayDetailViewModel. When HomeViewModel successfully loads the plant of the day it calls holder.set(plant), making the data available to the detail screen via holder.current(StateFlow).

6.5 SafetyAdvisor

A pure domain object that evaluates a PlantAiInfo.SafetyInfo against the user's SupabaseProfile. Returns a list of SafetyAlert objects which PlantDetailViewModel exposes as a StateFLow.

7. Testing

7.1 Test Stack

Tool	Type	Scope
Junit 4 + kotlinx-coroutines-test	Unit	ViewModels, repositories, services
MockK	Mocking	All unit tests
Compose UI Test	UI unit	All screens
Hilt	Instrumented UI	All screen needing real navigation + permission grants
JaCoCO	Coverage	Combined UI and Unit test coverage
GitHub Actions	CI	Lint + build + instrumented tests on API 35 emulator on pull request and push to main

7.2 Unit Test coverage

Coverage is measured with JaCoCo. Compose-generated bytecode is excluded so numbers reflect production logic only. The table below shows per package instruction coverage from the combined unit + instrumented test run.

Package com.plantsnap	Coverage	Notes
.ui.navigation	100%	Full nav route coverage
.ui.screens.history	98%	HistoryViewModel + HistoryScreen
.ui.screens.home	96%	HomeViewmodel + HomeScreen
.ui.screens.profile.model	93%	Profile model classes
.ui.state	93%	UiState sealed class
.data	90%	Core data layer
.ui.screens.identify.detail	89%	PlantDetailViewModel + PlantDetailScreen, remaining 11% is @Preview functions
.data.remote.supabase	85%	SupabaseProfile, Settings DTOs,

		mapping functions
.data.device	80%	DeviceIdProvider
.ui.screens.profile	54%	AuthViewModel covered, ProfileViewModel and ProfileScreen partially covered
.domain.models	47%	Domain data classes
.ui.screens.identify.camera	43%	CameScreenTest covers Ui, CameraViewModel logic partially covered
.data.repository	4%	GeminiRepositoryImpl tested via open subclass, not captured by JaCoCO
.data.local	17%	Room DAO implementations
.ui.screens.onboarding	0%	No unit tests, onboarding flow not prioritised in sprint scope since extra feature
.ui.screens.identify.identify	0%	IdentifyViewMOdel tested, IdentificationScreen Ui not yet covered
.data.plantnet	0%	PlantNet DTOs and mappers , no unit tests written

7.3 Running Tests

```
# Unit tests (JVM — no device required)
./gradlew testDebugUnitTest

# Coverage report — open app/build/reports/jacoco/jacocoTestReport/html/index.html
./gradlew testDebugUnitTest jacocoTestReport

# Instrumented UI tests (requires emulator or connected device)
./gradlew connectedDebugAndroidTest
```

7.4 Static Code Analysis - SonarCloud

SonarCloud is configured for the project and analyses the main branch on each push via GitHub Actions. The overall code report for the main branch is shown below.

Metric	Result	Grade
Security	0 open issues	A
Reliability	0 open issues	A
Maintainability	25 open issues	A
Duplications	0,3% (14k lines analysed)	-
Security Hotspots	4	-

The 25 maintainability issues are minor code smells, none are blocking defects. The security hotspots are flagged for review.

8. Build and Setup

8.1 Prerequisites

- Android studio Meerkat or later
- JDK 11
- Android emulator or physical device running API 26+
- Supabase project
- PlantNet API key - register at my.plantnet.org
- Google Gemini API key - create at aistudio.google.com
- Google OAuth client ID - configure in Google cloud console

8.2 API keys (local.properties)

Create a local.properties file in the project root. This file is gitignored and must never be committed.

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_KEY=your-supabase-key
GOOGLE_SERVER_CLIENT_ID=your-google-oauth-client-id
PLANTNET_API_KEY=your-plantnet-api-key
GOOGLE_API_KEY=your-gemini-api-key
MAPS_APIKEY=your-maps-api-key
```

8.3 Supabase setup

1. Create a new Supabase project at supabase.com
2. Run the profiles table SQL
3. Run the settings table SQL
4. Enable Google Auth provider in Supabase Dashboard
5. Add both debug and release SHA-1 fingerprints to Google CLoud Console OAuth client
6. Create a saved_plants Storage bucked in Supabase Storage

8.4 Build configuration

Property	Value
Language	Kotlin
Min SDK	26
Taget SDK	36
Compile SDK	36
Build tools	Gradle 9.2.1 + AGP 9.0.1

KSP	Kotlin Symbol processing- used by Hilt and Room
Serialization	kotlinx.serialization 1.7.3 - JSON for Supabase + Gemini responses

8.5 Key dependencies

Dependency	Version	Purpose
Hilt	2.89.2	Dependency injection
Room	2.8.4	Local SQLite database
CameraX	1.4.1	Camera lifecycle management
Retrofit	2.11.0	PlantNet API client
OkHttp	5.3.0	HTTP client
Coil	3.x	Image loading in compose
Supabase	3.1.1	Auth + cloud database + storage
DataStore preferences	1.1.1	Local settings cache
Google GenAI SDK	1.47.0	Gemini API client
Kotlinx Serialization	1.7.3	JSON serialization
Compose Navigation	-	Single-activity navigation

9 Continuous integration

A GitHub Actions pipeline runs on every pull request and push to main. The pipeline:

- Runs lint and compiles debug APK
- Executes instrumented tests and unit tests